

Assignment 05: Stacks, Queues, and Deques

Tyler Rex | CS 210 | San Diego State University | C++17

Overview

This program implements six integer abstract data types from scratch: an array stack, linked stack, array queue, linked queue, array deque, and linked deque. The code is split into `Stack.cpp`, `Queue.cpp`, `Deque.cpp`, a short `main.cpp`, and one header; it uses ordinary classes and functions without templates or auto variables. It uses no `<stack>`, `<queue>`, `<deque>`, or container that implements these structures. The three array versions have a fixed capacity of five so full behavior can be demonstrated.

Compile in the `Assignment05` directory with `clang++ -std=c++17 -Wall -Wextra main.cpp Stack.cpp Queue.cpp Deque.cpp -o assignment05`, then run `./assignment05`. The requested build completed with no warnings. A separate `AddressSanitizer` and `UndefinedBehaviorSanitizer` run also finished with zero failures.

Design and empty handling

Stacks

The array stack stores its bottom-to-top values in `data[0]` through `data[count - 1]`. `push` writes at `count` and increments it; `pop` decrements `count` and returns that element. The linked stack keeps the top at `head`. `push` adds a node before `head`, and `pop` removes `head`. Both therefore return the newest value first (LIFO).

Queues

The array queue uses `front` and `count` as circular-array state. `enqueue` writes to $(\text{front} + \text{count}) \% \text{CAPACITY}$. `dequeue` reads `front` and advances it with modulo; no shifting is needed. The linked queue keeps `head` for the oldest item and `tail` for the newest. `enqueue` links after `tail`, and `dequeue` removes `head`. When the last node is removed, `tail` becomes `nullptr`. Both preserve FIFO order.

Dequeues

The array deque uses the same circular indexing. `pushFront` moves `front` backward before writing; `pushBack` writes at $(\text{front} + \text{count}) \% \text{CAPACITY}$. `popFront` advances `front`; `popBack` reads the last occupied index and decrements `count`. The linked deque is doubly linked: `head` and `tail` are the two ends, and each node has `previous` and `next` pointers. This allows all four end operations without traversal.

All removal functions return `false` on an empty structure and leave the output parameter unchanged. Full fixed arrays reject insertion and leave their contents unchanged. Linked classes delete remaining nodes in their destructors. The program never copies a linked structure.

Time and space complexity

Implementation	Insertion	Removal	Check / size	Space
Array stack	$O(1)$	$O(1)$	$O(1)$	$O(C)$
Linked stack	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Array queue	$O(1)$	$O(1)$	$O(1)$	$O(C)$
Linked queue	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Array deque	$O(1)$ both ends	$O(1)$ both ends	$O(1)$	$O(C)$
Linked deque	$O(1)$ both ends	$O(1)$ both ends	$O(1)$	$O(n)$

Here n is the current item count and $C = 5$ is the fixed array capacity. The operations in this particular design never become $O(n)$: a full array insertion returns false, circular indices prevent shifting, and the linked deque has previous pointers. A resizable array would have an $O(n)$ growth step; a shifting array queue or a singly linked deque popBack could take $O(n)$. Those are comparison designs, not operations used here. Destruction of n remaining linked nodes is $O(n)$. Visiting or printing every stored item would also be $O(n)$.

Tests and actual results

The demonstration functions check each attempted insertion or removal against an expected success result and, for successful removals, an expected value. It also checks that failed removals leave the output value unchanged. It tests empty removals before and after use, array overflow, LIFO order, FIFO order, both deque ends, circular wraparound, and an empty final size. Every structure has at least 16 operations: array stack 17, linked stack 16, array queue 17, linked queue 16, array deque 19, and linked deque 16. The actual run recorded 101 operations and 0 failures. The complete captured console output follows.

```
ARRAY STACK
1. pop -> empty [PASS]
2. push 10 -> success [PASS]
3. push 20 -> success [PASS]
4. push 30 -> success [PASS]
5. pop -> 30 [PASS]
6. push 40 -> success [PASS]
7. push 50 -> success [PASS]
8. push 60 -> success [PASS]
9. push 99 while full -> rejected [PASS]
10. pop -> 60 [PASS]
11. pop -> 50 [PASS]
12. push 70 -> success [PASS]
13. pop -> 70 [PASS]
14. pop -> 40 [PASS]
15. pop -> 20 [PASS]
16. pop -> 10 [PASS]
17. pop -> empty [PASS]
Operations: 17; final size: 0
```

```
LINKED STACK
1. pop -> empty [PASS]
2. push 10 -> success [PASS]
3. push 20 -> success [PASS]
4. push 30 -> success [PASS]
5. pop -> 30 [PASS]
6. push 40 -> success [PASS]
7. push 50 -> success [PASS]
8. push 60 -> success [PASS]
9. pop -> 60 [PASS]
10. pop -> 50 [PASS]
11. push 70 -> success [PASS]
12. pop -> 70 [PASS]
13. pop -> 40 [PASS]
14. pop -> 20 [PASS]
15. pop -> 10 [PASS]
16. pop -> empty [PASS]
Operations: 16; final size: 0
```

```
ARRAY QUEUE
```

```

1. dequeue -> empty [PASS]
2. enqueue 10 -> success [PASS]
3. enqueue 20 -> success [PASS]
4. enqueue 30 -> success [PASS]
5. enqueue 40 -> success [PASS]
6. enqueue 50 -> success [PASS]
7. enqueue 99 while full -> rejected [PASS]
8. dequeue -> 10 [PASS]
9. dequeue -> 20 [PASS]
10. enqueue 60 -> success [PASS]
11. enqueue 70 -> success [PASS]
12. dequeue -> 30 [PASS]
13. dequeue -> 40 [PASS]
14. dequeue -> 50 [PASS]
15. dequeue -> 60 [PASS]
16. dequeue -> 70 [PASS]
17. dequeue -> empty [PASS]
Operations: 17; final size: 0

```

LINKED QUEUE

```

1. dequeue -> empty [PASS]
2. enqueue 10 -> success [PASS]
3. enqueue 20 -> success [PASS]
4. enqueue 30 -> success [PASS]
5. enqueue 40 -> success [PASS]
6. enqueue 50 -> success [PASS]
7. dequeue -> 10 [PASS]
8. dequeue -> 20 [PASS]
9. enqueue 60 -> success [PASS]
10. enqueue 70 -> success [PASS]
11. dequeue -> 30 [PASS]
12. dequeue -> 40 [PASS]
13. dequeue -> 50 [PASS]
14. dequeue -> 60 [PASS]
15. dequeue -> 70 [PASS]
16. dequeue -> empty [PASS]
Operations: 16; final size: 0

```

ARRAY DEQUE

```

1. popFront -> empty [PASS]
2. pushBack 10 -> success [PASS]
3. pushFront 20 -> success [PASS]
4. pushBack 30 -> success [PASS]
5. popBack -> 30 [PASS]
6. pushFront 40 -> success [PASS]
7. pushBack 50 -> success [PASS]
8. popFront -> 40 [PASS]
9. pushBack 60 -> success [PASS]
10. popBack -> 60 [PASS]
11. pushFront 70 -> success [PASS]
12. pushBack 80 -> success [PASS]
13. pushFront 99 while full -> rejected [PASS]
14. popBack -> 80 [PASS]
15. popFront -> 70 [PASS]
16. popFront -> 20 [PASS]
17. popBack -> 50 [PASS]
18. popBack -> 10 [PASS]
19. popBack -> empty [PASS]
Operations: 19; final size: 0

```

LINKED DEQUE

```

1. popFront -> empty [PASS]
2. pushBack 10 -> success [PASS]
3. pushFront 20 -> success [PASS]
4. pushBack 30 -> success [PASS]
5. popBack -> 30 [PASS]
6. pushFront 40 -> success [PASS]
7. pushBack 50 -> success [PASS]
8. popFront -> 40 [PASS]
9. pushBack 60 -> success [PASS]
10. popBack -> 60 [PASS]
11. pushFront 70 -> success [PASS]
12. popFront -> 70 [PASS]
13. popFront -> 20 [PASS]
14. popBack -> 50 [PASS]
15. popBack -> 10 [PASS]
16. popBack -> empty [PASS]
Operations: 16; final size: 0

```

TOTAL: 101 operations; 0 failures

Complete course-relevant source code

All five files below were used to compile the program and produce the output above.

Assignment05.h

```
#ifndef ASSIGNMENT05_H
#define ASSIGNMENT05_H

#include <string>

using namespace std;

// These functions print one operation and compare it with the expected result.
void checkInsert(string action, bool actual, bool expected,
                int& steps, int& failures);
void checkRemove(string action, bool actual, int value,
                bool expected, int expectedValue, int& steps, int& failures);

// Each function demonstrates the array and linked versions of one ADT.
void testStacks(int& totalSteps, int& failures);
void testQueues(int& totalSteps, int& failures);
void testDequeues(int& totalSteps, int& failures);

#endif
```

Stack.cpp

```
#include "Assignment05.h"
#include <iostream>

using namespace std;

const int STACK_CAPACITY = 5;

class ArrayStack {
private:
    int data[STACK_CAPACITY];
    int count;

public:
    ArrayStack() { count = 0; }

    bool push(int value) {
        if (count == STACK_CAPACITY) return false;
        data[count] = value;
        count++;
        return true;
    }

    bool pop(int& value) {
        if (count == 0) return false;
        count--;
        value = data[count];
        return true;
    }

    bool isEmpty() { return count == 0; }
    int size() { return count; }
};

class LinkedStack {
private:
    struct Node {
        int value;
        Node* next;
    };
    Node* head;
    int count;

public:
    LinkedStack() {
        head = nullptr;
        count = 0;
    }

    bool push(int value) {
        Node* node = new Node;
        node->value = value;
        node->next = head;
        head = node;
        count++;
        return true;
    }

    bool pop(int& value) {
        if (head == nullptr) return false;
        Node* old = head;
        value = old->value;
        head = head->next;
        delete old;
        count--;
        return true;
    }

    bool isEmpty() { return head == nullptr; }
    int size() { return count; }

    ~LinkedStack() {
        while (head != nullptr) {
            Node* old = head;
            head = head->next;
            delete old;
        }
    }
};

void testStacks(int& totalSteps, int& failures) {
    int steps = 0;

    int value;
    bool success;

    cout << "\nARRAY STACK" << endl;
    ArrayStack arrayStack;
```

```

value = -1; // Failed removal must not change this value.
success = arrayStack.pop(value);
checkRemove("pop", success, value, false, -1, steps, failures);
success = arrayStack.push(10);
checkInsert("push 10", success, true, steps, failures);
success = arrayStack.push(20);
checkInsert("push 20", success, true, steps, failures);
success = arrayStack.push(30);
checkInsert("push 30", success, true, steps, failures);
success = arrayStack.pop(value);
checkRemove("pop", success, value, true, 30, steps, failures);
success = arrayStack.push(40);
checkInsert("push 40", success, true, steps, failures);
success = arrayStack.push(50);
checkInsert("push 50", success, true, steps, failures);
success = arrayStack.push(60);
checkInsert("push 60", success, true, steps, failures);
success = arrayStack.push(99);
checkInsert("push 99 while full", success, false, steps, failures);
success = arrayStack.pop(value);
checkRemove("pop", success, value, true, 60, steps, failures);
success = arrayStack.pop(value);
checkRemove("pop", success, value, true, 50, steps, failures);
success = arrayStack.push(70);
checkInsert("push 70", success, true, steps, failures);
success = arrayStack.pop(value);
checkRemove("pop", success, value, true, 70, steps, failures);
success = arrayStack.pop(value);
checkRemove("pop", success, value, true, 40, steps, failures);
success = arrayStack.pop(value);
checkRemove("pop", success, value, true, 20, steps, failures);
success = arrayStack.pop(value);
checkRemove("pop", success, value, true, 10, steps, failures);
value = -1; // Failed removal must not change this value.
success = arrayStack.pop(value);
checkRemove("pop", success, value, false, -1, steps, failures);
if (!arrayStack.isEmpty() || arrayStack.size() != 0) failures++;
cout << "Operations: " << steps << "; final size: " << arrayStack.size() << endl;
totalSteps += steps;

cout << "\nLINKED STACK" << endl;
LinkedList linkedStack;
steps = 0;
value = -1; // Failed removal must not change this value.
success = linkedStack.pop(value);
checkRemove("pop", success, value, false, -1, steps, failures);
success = linkedStack.push(10);
checkInsert("push 10", success, true, steps, failures);
success = linkedStack.push(20);
checkInsert("push 20", success, true, steps, failures);
success = linkedStack.push(30);
checkInsert("push 30", success, true, steps, failures);
success = linkedStack.pop(value);
checkRemove("pop", success, value, true, 30, steps, failures);
success = linkedStack.push(40);
checkInsert("push 40", success, true, steps, failures);
success = linkedStack.push(50);
checkInsert("push 50", success, true, steps, failures);
success = linkedStack.push(60);
checkInsert("push 60", success, true, steps, failures);
success = linkedStack.pop(value);
checkRemove("pop", success, value, true, 60, steps, failures);
success = linkedStack.pop(value);
checkRemove("pop", success, value, true, 50, steps, failures);
success = linkedStack.push(70);
checkInsert("push 70", success, true, steps, failures);
success = linkedStack.pop(value);
checkRemove("pop", success, value, true, 70, steps, failures);
success = linkedStack.pop(value);
checkRemove("pop", success, value, true, 40, steps, failures);
success = linkedStack.pop(value);
checkRemove("pop", success, value, true, 20, steps, failures);
success = linkedStack.pop(value);
checkRemove("pop", success, value, true, 10, steps, failures);
value = -1; // Failed removal must not change this value.
success = linkedStack.pop(value);

checkRemove("pop", success, value, false, -1, steps, failures);
if (!linkedStack.isEmpty() || linkedStack.size() != 0) failures++;
cout << "Operations: " << steps << "; final size: " << linkedStack.size() << endl;
totalSteps += steps;
}

```

Queue.cpp

```
#include "Assignment05.h"
#include <iostream>

using namespace std;

const int QUEUE_CAPACITY = 5;

class ArrayQueue {
private:
    int data[QUEUE_CAPACITY];
    int front;
    int count;

public:
    ArrayQueue() {
        front = 0;
        count = 0;
    }

    bool enqueue(int value) {
        if (count == QUEUE_CAPACITY) return false;
        int back = (front + count) % QUEUE_CAPACITY;
        data[back] = value;
        count++;
        return true;
    }

    bool dequeue(int& value) {
        if (count == 0) return false;
        value = data[front];
        front = (front + 1) % QUEUE_CAPACITY;
        count--;
        return true;
    }

    bool isEmpty() { return count == 0; }
    int size() { return count; }
};

class LinkedQueue {
private:
    struct Node {
        int value;
        Node* next;
    };
    Node* head;
    Node* tail;
    int count;

public:
    LinkedQueue() {
        head = nullptr;
        tail = nullptr;
        count = 0;
    }

    bool enqueue(int value) {
        Node* node = new Node;
        node->value = value;
        node->next = nullptr;

        if (tail == nullptr) head = node;
        else tail->next = node;
        tail = node;
        count++;
        return true;
    }

    bool dequeue(int& value) {
        if (head == nullptr) return false;
        Node* old = head;
        value = old->value;
        head = head->next;
        if (head == nullptr) tail = nullptr;
        delete old;
        count--;
        return true;
    }

    bool isEmpty() { return head == nullptr; }
    int size() { return count; }

    ~LinkedQueue() {
        while (head != nullptr) {
            Node* old = head;
            head = head->next;
            delete old;
        }
    }
};
```

```

    }
}
};

void testQueues(int& totalSteps, int& failures) {
    int steps = 0;
    int value;
    bool success;

    cout << "\nARRAY QUEUE" << endl;
    ArrayQueue arrayQueue;
    value = -1; // Failed removal must not change this value.
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, false, -1, steps, failures);
    success = arrayQueue.enqueue(10);
    checkInsert("enqueue 10", success, true, steps, failures);
    success = arrayQueue.enqueue(20);
    checkInsert("enqueue 20", success, true, steps, failures);
    success = arrayQueue.enqueue(30);
    checkInsert("enqueue 30", success, true, steps, failures);
    success = arrayQueue.enqueue(40);
    checkInsert("enqueue 40", success, true, steps, failures);
    success = arrayQueue.enqueue(50);
    checkInsert("enqueue 50", success, true, steps, failures);
    success = arrayQueue.enqueue(99);
    checkInsert("enqueue 99 while full", success, false, steps, failures);
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 10, steps, failures);
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 20, steps, failures);
    success = arrayQueue.enqueue(60);
    checkInsert("enqueue 60", success, true, steps, failures);
    success = arrayQueue.enqueue(70);
    checkInsert("enqueue 70", success, true, steps, failures);
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 30, steps, failures);
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 40, steps, failures);
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 50, steps, failures);
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 60, steps, failures);
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 70, steps, failures);
    value = -1; // Failed removal must not change this value.
    success = arrayQueue.dequeue(value);
    checkRemove("dequeue", success, value, false, -1, steps, failures);
    if (!arrayQueue.isEmpty() || arrayQueue.size() != 0) failures++;
    cout << "Operations: " << steps << "; final size: " << arrayQueue.size() << endl;
    totalSteps += steps;

    cout << "\nLINKED QUEUE" << endl;
    LinkedQueue linkedQueue;
    steps = 0;
    value = -1; // Failed removal must not change this value.
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, false, -1, steps, failures);
    success = linkedQueue.enqueue(10);
    checkInsert("enqueue 10", success, true, steps, failures);
    success = linkedQueue.enqueue(20);
    checkInsert("enqueue 20", success, true, steps, failures);
    success = linkedQueue.enqueue(30);
    checkInsert("enqueue 30", success, true, steps, failures);
    success = linkedQueue.enqueue(40);
    checkInsert("enqueue 40", success, true, steps, failures);
    success = linkedQueue.enqueue(50);
    checkInsert("enqueue 50", success, true, steps, failures);
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 10, steps, failures);
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 20, steps, failures);
    success = linkedQueue.enqueue(60);
    checkInsert("enqueue 60", success, true, steps, failures);
    success = linkedQueue.enqueue(70);
    checkInsert("enqueue 70", success, true, steps, failures);

    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 30, steps, failures);
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 40, steps, failures);
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 50, steps, failures);
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 60, steps, failures);
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, true, 70, steps, failures);
    value = -1; // Failed removal must not change this value.
    success = linkedQueue.dequeue(value);
    checkRemove("dequeue", success, value, false, -1, steps, failures);

```

```
if (!linkedQueue.isEmpty() || linkedQueue.size() != 0) failures++;  
cout << "Operations: " << steps << "; final size: " << linkedQueue.size() << endl;  
totalSteps += steps;  
}
```

Deque.cpp

```
#include "Assignment05.h"
#include <iostream>

using namespace std;

const int DEQUE_CAPACITY = 5;

class ArrayDeque {
private:
    int data[DEQUE_CAPACITY];
    int front;
    int count;

public:
    ArrayDeque() {
        front = 0;
        count = 0;
    }

    bool pushFront(int value) {
        if (count == DEQUE_CAPACITY) return false;
        front = (front + DEQUE_CAPACITY - 1) % DEQUE_CAPACITY;
        data[front] = value;
        count++;
        return true;
    }

    bool pushBack(int value) {
        if (count == DEQUE_CAPACITY) return false;
        int back = (front + count) % DEQUE_CAPACITY;
        data[back] = value;
        count++;
        return true;
    }

    bool popFront(int& value) {
        if (count == 0) return false;
        value = data[front];
        front = (front + 1) % DEQUE_CAPACITY;
        count--;
        return true;
    }

    bool popBack(int& value) {
        if (count == 0) return false;
        int back = (front + count - 1) % DEQUE_CAPACITY;
        value = data[back];
        count--;
        return true;
    }

    bool isEmpty() { return count == 0; }
    int size() { return count; }
};

class LinkedDeque {
private:
    struct Node {
        int value;
        Node* previous;
        Node* next;
    };
    Node* head;
    Node* tail;
    int count;

public:
    LinkedDeque() {
        head = nullptr;
        tail = nullptr;
        count = 0;
    }

    bool pushFront(int value) {
        Node* node = new Node;
        node->value = value;
        node->previous = nullptr;
        node->next = head;
        if (head == nullptr) tail = node;
        else head->previous = node;
        head = node;

        count++;
        return true;
    }

    bool pushBack(int value) {
```

```

        Node* node = new Node;
        node->value = value;
        node->previous = tail;
        node->next = nullptr;
        if (tail == nullptr) head = node;
        else tail->next = node;
        tail = node;
        count++;
        return true;
    }

    bool popFront(int& value) {
        if (head == nullptr) return false;
        Node* old = head;
        value = old->value;
        head = head->next;
        if (head == nullptr) tail = nullptr;
        else head->previous = nullptr;
        delete old;
        count--;
        return true;
    }

    bool popBack(int& value) {
        if (tail == nullptr) return false;
        Node* old = tail;
        value = old->value;
        tail = tail->previous;
        if (tail == nullptr) head = nullptr;
        else tail->next = nullptr;
        delete old;
        count--;
        return true;
    }

    bool isEmpty() { return head == nullptr; }
    int size() { return count; }

    ~LinkedList() {
        while (head != nullptr) {
            Node* old = head;
            head = head->next;
            delete old;
        }
    }
};

void testDeque(int& totalSteps, int& failures) {
    int steps = 0;
    int value;
    bool success;

    cout << "\nARRAY DEQUE" << endl;
    ArrayDeque arrayDeque;
    value = -1; // Failed removal must not change this value.
    success = arrayDeque.popFront(value);
    checkRemove("popFront", success, value, false, -1, steps, failures);
    success = arrayDeque.pushBack(10);
    checkInsert("pushBack 10", success, true, steps, failures);
    success = arrayDeque.pushFront(20);
    checkInsert("pushFront 20", success, true, steps, failures);
    success = arrayDeque.pushBack(30);
    checkInsert("pushBack 30", success, true, steps, failures);
    success = arrayDeque.popBack(value);
    checkRemove("popBack", success, value, true, 30, steps, failures);
    success = arrayDeque.pushFront(40);
    checkInsert("pushFront 40", success, true, steps, failures);
    success = arrayDeque.pushBack(50);
    checkInsert("pushBack 50", success, true, steps, failures);
    success = arrayDeque.popFront(value);
    checkRemove("popFront", success, value, true, 40, steps, failures);
    success = arrayDeque.pushBack(60);
    checkInsert("pushBack 60", success, true, steps, failures);
    success = arrayDeque.popBack(value);
    checkRemove("popBack", success, value, true, 60, steps, failures);
    success = arrayDeque.pushFront(70);
    checkInsert("pushFront 70", success, true, steps, failures);
    success = arrayDeque.pushBack(80);
    checkInsert("pushBack 80", success, true, steps, failures);
    success = arrayDeque.pushFront(99);
    checkInsert("pushFront 99 while full", success, false, steps, failures);
    success = arrayDeque.popBack(value);
    checkRemove("popBack", success, value, true, 80, steps, failures);
    success = arrayDeque.popFront(value);
    checkRemove("popFront", success, value, true, 70, steps, failures);
    success = arrayDeque.popFront(value);
    checkRemove("popFront", success, value, true, 20, steps, failures);
    success = arrayDeque.popBack(value);

```

```

checkRemove("popBack", success, value, true, 50, steps, failures);
success = arrayDeque.popBack(value);
checkRemove("popBack", success, value, true, 10, steps, failures);
value = -1; // Failed removal must not change this value.
success = arrayDeque.popBack(value);
checkRemove("popBack", success, value, false, -1, steps, failures);
if (!arrayDeque.isEmpty() || arrayDeque.size() != 0) failures++;
cout << "Operations: " << steps << "; final size: " << arrayDeque.size() << endl;
totalSteps += steps;

cout << "\nLINKED DEQUE" << endl;
LinkedList linkedDeque;
steps = 0;
value = -1; // Failed removal must not change this value.
success = linkedDeque.popFront(value);
checkRemove("popFront", success, value, false, -1, steps, failures);
success = linkedDeque.pushBack(10);
checkInsert("pushBack 10", success, true, steps, failures);
success = linkedDeque.pushFront(20);
checkInsert("pushFront 20", success, true, steps, failures);
success = linkedDeque.pushBack(30);
checkInsert("pushBack 30", success, true, steps, failures);
success = linkedDeque.popBack(value);
checkRemove("popBack", success, value, true, 30, steps, failures);
success = linkedDeque.pushFront(40);
checkInsert("pushFront 40", success, true, steps, failures);
success = linkedDeque.pushBack(50);
checkInsert("pushBack 50", success, true, steps, failures);
success = linkedDeque.popFront(value);
checkRemove("popFront", success, value, true, 40, steps, failures);
success = linkedDeque.pushBack(60);
checkInsert("pushBack 60", success, true, steps, failures);
success = linkedDeque.popBack(value);
checkRemove("popBack", success, value, true, 60, steps, failures);
success = linkedDeque.pushFront(70);
checkInsert("pushFront 70", success, true, steps, failures);
success = linkedDeque.popFront(value);
checkRemove("popFront", success, value, true, 70, steps, failures);
success = linkedDeque.popFront(value);
checkRemove("popFront", success, value, true, 20, steps, failures);
success = linkedDeque.popBack(value);
checkRemove("popBack", success, value, true, 50, steps, failures);
success = linkedDeque.popBack(value);
checkRemove("popBack", success, value, true, 10, steps, failures);
value = -1; // Failed removal must not change this value.
success = linkedDeque.popBack(value);
checkRemove("popBack", success, value, false, -1, steps, failures);
if (!linkedDeque.isEmpty() || linkedDeque.size() != 0) failures++;
cout << "Operations: " << steps << "; final size: " << linkedDeque.size() << endl;
totalSteps += steps;
}

```

main.cpp

```
#include "Assignment05.h"
#include <iostream>

using namespace std;

void checkInsert(string action, bool actual, bool expected,
                int& steps, int& failures) {
    steps++;
    bool passed = (actual == expected);
    if (!passed) failures++;

    cout << steps << ". " << action << " -> ";
    if (actual) cout << "success";
    else cout << "rejected";
    if (passed) cout << " [PASS]" << endl;
    else cout << " [FAIL]" << endl;
}

void checkRemove(string action, bool actual, int value,
                bool expected, int expectedValue, int& steps, int& failures) {
    steps++;
    bool passed = (actual == expected && value == expectedValue);
    if (!passed) failures++;

    cout << steps << ". " << action << " -> ";
    if (actual) cout << value;
    else cout << "empty";
    if (passed) cout << " [PASS]" << endl;
    else cout << " [FAIL]" << endl;
}

int main() {
    int totalSteps = 0;
    int failures = 0;

    testStacks(totalSteps, failures);
    testQueues(totalSteps, failures);
    testDequeues(totalSteps, failures);

    cout << "\nTOTAL: " << totalSteps << " operations; "
         << failures << " failures" << endl;

    if (failures == 0) return 0;
    return 1;
}
```